



문제 해결을 위한 알고리즘 연습

문제 두 정수의 최대공약수를 구하여 결과를 출력해 봅시다. (유클리드 호제법)

🕒 문제 해결 방법

유클리드 호제법에 따르면 피연산자를 순환하면서 나머지 연산을 하는 동작을 피연산자가 0이 될 때까지 반복하면 최대공약수를 얻을 수 있습니다.

※ 관련 알고리즘 : 피연산자를 순환하여 연산하기(29쪽 참고)

[입력] 정수 2개

[출력] 정수 2개의 최대공약수

다음의 예를 통해 내용을 이해해 봅시다.

첫 번째 숫자

8652

두 번째 숫자

2766

최대공약수

6

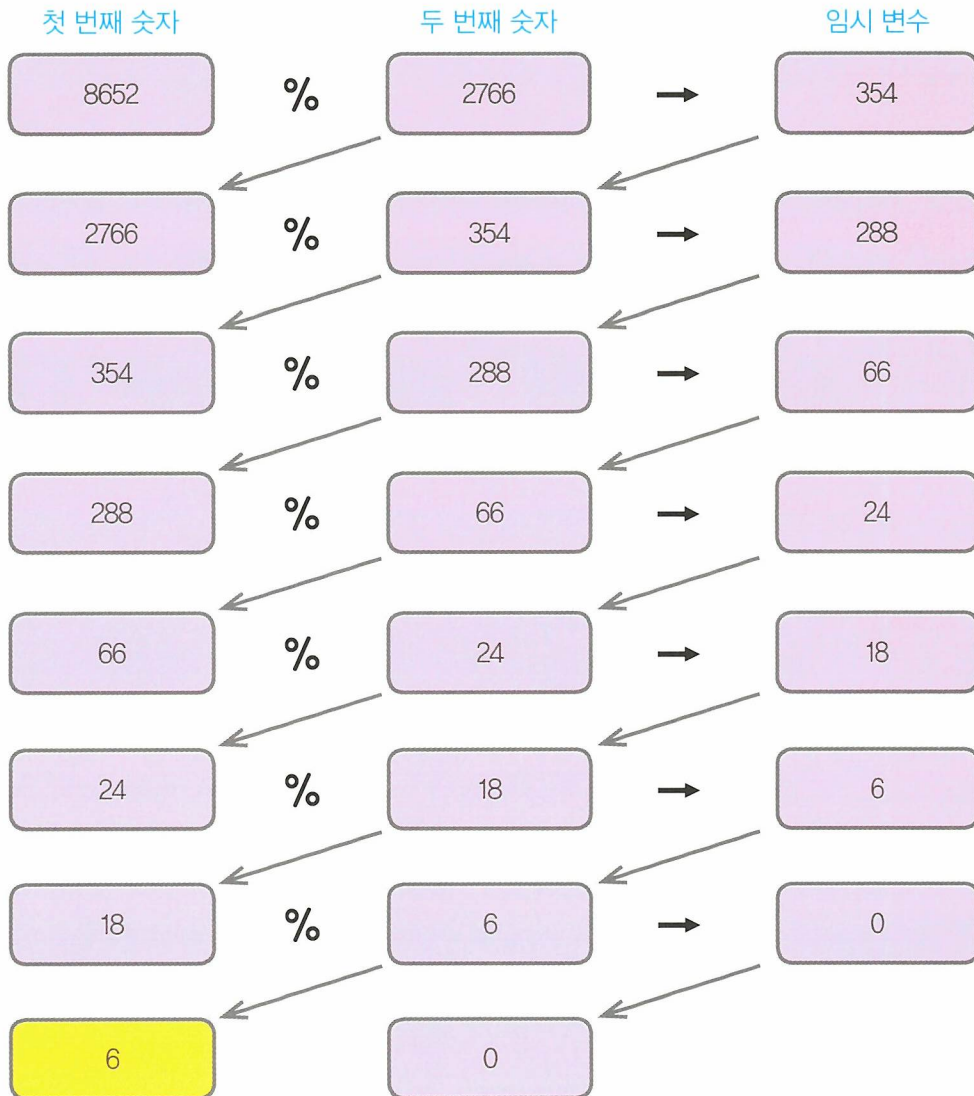
다음과 같은 순서로 문제를 해결할 수 있습니다.

1. 순환되는 피연산자가 0이 될 때까지 다음을 반복합니다.
 - 1.1 피연산자를 순환하여 나머지 연산을 합니다.
2. 결과를 반환합니다.



문제 해결의 예

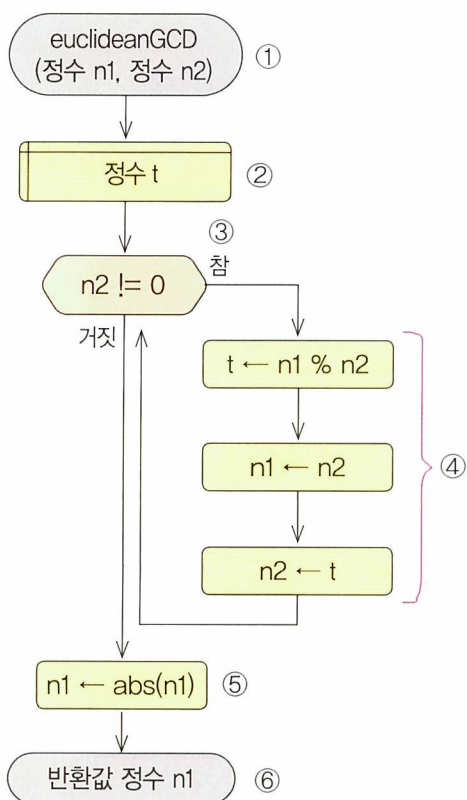
|예| 8652와 2766의 최대공약수 구하는 경우



→ 실행한 결과로 8652와 2766의 최대공약수는 6이 출력됩니다.



순서도와 알고리즘 해설(※ 142쪽 주프로그램 참고)



① 함수를 정의합니다.

- 함수명은 euclideanGCD입니다.

- 변수 n1, n2는 정수를 전달 받는 매개변수들입니다.

② 이 알고리즘에서 필요한 변수는 다음과 같습니다.

- 변수 t는 중간 결과를 저장하기 위한 임시 변수입니다.

③ 반복 조건을 설정합니다.

- 변수 n2의 값이 0이 될 때까지 반복합니다.

④ 피연산자를 순환하는 나머지 연산을 합니다.

- 두 번째 피연산자의 값을 첫 번째 피연산자를 저장하는 변수 n1로, 나머지 연산을 한 결과를 두 번째 피연산자를 저장하는 변수 n2로 이동합니다.(※ 29쪽 참고)

⑤ 최대공약수는 자연수이므로 절대값으로 변환합니다.

- 변수 n1의 값을 절대값으로 변환합니다.

⑥ 최대공약수를 반환합니다.

- 변수 n1의 값을 반환합니다.



문제 정해진 범위 내의 소수를 구하여 결과를 출력해 봅시다. (에라토스테네스의 체)

🕒 문제 해결 방법

0부터 19까지의 범위 내에서 소수를 찾기 위해 첨자가 19까지인 배열을 만듭니다. 2의 2배 이상의 배수의 첨자가 가리키는 배열 요소를 표시하고 (실제는 '1'을 저장), 다음에는 3의 2배 이상의 배수가 첨자로 가리키는 배열 요소를 표시합니다. 이러한 과정을 주어진 정수(여기에서는 19)의 제곱근(여기에서는 19의 제곱근이 4.3588... 이므로 4)까지 반복합니다.

1이 저장된 배열을 제외하고 나머지 배열을 걸러내면 해당 배열의 첨자가 소수들입니다 (단, 첨자 0과 1은 제외). 이러한 방법을 '에라토스테네스의 체 (Sieve of Eratosthenes)'라고 부릅니다.

※ 관련 알고리즘 : 주어진 숫자의 배수 표시하기 (73쪽 참고)

[입력] 정수

[출력] 소수가 가리키는 배열 요소 값이 1인 배열

0	1	2	3	4	5	6	7	8	9	10	11	12	13	14	15	16	17	18	19
0	0	0	0	1	0	1	0	1	1	1	0	1	0	1	1	1	0	1	0

다음과 같은 순서로 문제를 해결할 수 있습니다.

- 2부터 주어진 숫자의 제곱근까지 1씩 증가시키면서 다음을 반복합니다.
 - 현재 숫자의 2배 이상의 배수를 표시합니다.
- 소수를 가리키는 배열 요소 값이 1로 저장되어 있는 배열을 반환합니다.



문제 해결의 예

[예] 0~19 사이의 소수 찾기**처음 상태 :** 19의 제곱근은 4.3588... 이므로 i가 4가 될 때까지 반복합니다.

0	1	2	3	4	5	6	7	8	9	10	11	12	13	14	15	16	17	18	19
0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0

1번째 반복 : 2의 2배 이상의 배수인 4, 6, 8, 10, 12, 14, 16, 18이 첨자인 곳에 1을 저장합니다.

0	1	2	3	4	5	6	7	8	9	10	11	12	13	14	15	16	17	18	19
0	0	0	0	1	0	1	0	1	0	1	0	1	0	1	0	1	0	1	0

2번째 반복 : 3의 2배 이상의 배수인 6, 9, 12, 15, 18이 첨자인 곳에 1을 저장합니다.

0	1	2	3	4	5	6	7	8	9	10	11	12	13	14	15	16	17	18	19
0	0	0	0	0	0	1	0	0	1	0	0	1	0	0	1	0	0	1	0

3번째 반복 : 4의 2배 이상의 배수인 8, 12, 16이 첨자인 곳에 1을 저장합니다.

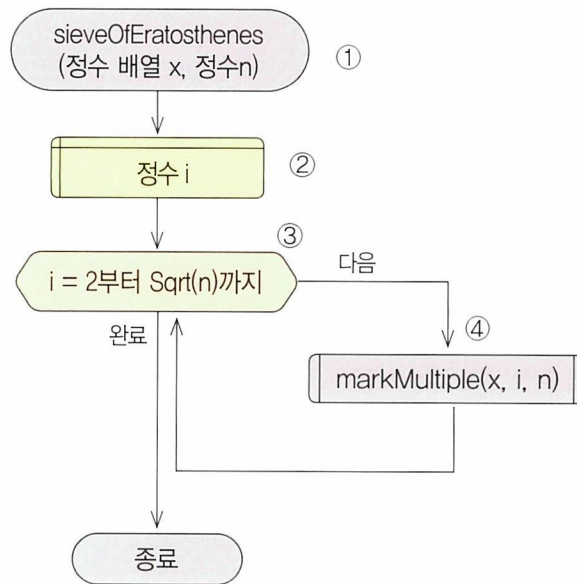
0	1	2	3	4	5	6	7	8	9	10	11	12	13	14	15	16	17	18	19
0	0	0	0	0	0	0	0	1	0	0	0	1	0	0	0	1	0	0	0

결과 : 첨자가 2 이상이며 값이 0인 요소의 첨자를 출력하면 다음과 같습니다. 2, 3, 5, 7, 11, 13, 17, 19

0	1	2	3	4	5	6	7	8	9	10	11	12	13	14	15	16	17	18	19
0	0	0	0	1	0	1	0	1	1	1	0	1	0	1	1	1	0	1	0



순서도와 알고리즘 해설(※ 143쪽 주프로그램 참고)



① 함수를 정의합니다.

- 함수명은 sieveOfEratosthenes입니다.
- 배열 x는 모든 요소에 0이 저장되어 있습니다.
- 변수 n은 배열의 범위를 지정합니다.

② 이 알고리즘에서 필요한 변수는 다음과 같습니다.

- 변수 i는 반복을 제어하기 위한 변수입니다.

③ 반복 횟수를 설정합니다.

- 변수 i를 2부터 변수 n에 저장된 숫자의 제곱근까지 1씩 증가시켜 반복합니다.

④ 변수 i값의 배수를 표시합니다.

- 배열 x에서 변수 i의 배수와 같은 점자를 가진 배열에 1을 저장합니다.(단, 0과 1 제외)(※ 73쪽 참고)

※ 결과 배열에서 값이 0인 요소의 점자들이 소수이므로 이를 출력합니다.



문제 배열에 저장된 숫자의 순위를 정해 봅시다. 단, 가장 큰 숫자의 순위가 1이 됩니다.

🕒 문제 해결 방법

배열의 첫 번째 요소부터 차례대로 기준 숫자로 정해 배열의 모든 숫자에 대하여 기준 숫자보다 작거나 같은 값을 가진 위치의 요소에 1을 더하면 각각의 요소에 대한 순위가 표시된 배열을 구할 수 있습니다.

※ 관련 알고리즘 : 주어진 숫자보다 작거나 같은 숫자 표시하기 (76쪽 참고)

[입력] 숫자들이 저장되어 있는 배열

[출력] 주어진 배열의 각 요소들에 대한 순위가 저장된 배열

배열

23	41	25	87	33	96	26	74	51	84
----	----	----	----	----	----	----	----	----	----



결과

23	41	25	87	33	96	26	74	51	84
10	6	9	2	7	1	8	4	5	3

다음과 같은 순서로 문제를 해결할 수 있습니다.

- 배열의 모든 요소에 대하여 다음을 반복합니다.
 - 현재 요소의 값보다 작거나 같은 배열 요소의 순위를 1만큼 증가시킵니다.
- 순위가 저장된 배열을 반환합니다.



🔒 문제 해결의 예

|예| 10개의 요소를 가진 배열의 요소에 순위 매기기

1번째 반복 : 1번째 숫자인 23보다 작거나 같은 숫자가 있는 위치의 별도 배열의 요소 값을 1만큼 증가시킵니다.

23	41	25	87	33	96	26	74	51	84
1	0	0	0	0	0	0	0	0	0

2번째 반복 : 2번째 숫자인 41보다 작거나 같은 숫자가 있는 위치의 별도 배열의 요소 값을 1만큼 증가시킵니다.

23	41	25	87	33	96	26	74	51	84
2	1	1	0	1	0	1	0	0	0

3번째 반복 : 3번째 숫자인 25보다 작거나 같은 숫자가 있는 위치의 별도 배열의 요소 값을 1만큼 증가시킵니다.

23	41	25	87	33	96	26	74	51	84
3	1	2	0	1	0	1	0	0	0

4번째 반복 : 4번째 숫자인 87보다 작거나 같은 숫자가 있는 위치의 별도 배열의 요소 값을 1만큼 증가시킵니다.

23	41	25	87	33	96	26	74	51	84
4	2	3	1	2	0	2	1	1	1

5번째 반복 : 5번째 숫자인 33보다 작거나 같은 숫자가 있는 위치의 별도 배열의 요소 값을 1만큼 증가시킵니다.

23	41	25	87	33	96	26	74	51	84
5	2	4	1	3	0	3	1	1	1

6번째 반복 : 6번째 숫자인 96보다 작거나 같은 숫자가 있는 위치의 별도 배열의 요소 값을 1만큼 증가시킵니다.

23	41	25	87	33	96	26	74	51	84
6	3	5	2	4	1	4	2	2	2

7번째 반복 : 7번째 숫자인 26보다 작거나 같은 숫자가 있는 위치의 별도 배열의 요소 값을 1만큼 증가시킵니다.

23	41	25	87	33	96	26	74	51	84
7	3	6	2	4	1	5	2	2	2

8번째 반복 : 8번째 숫자인 74보다 작거나 같은 숫자가 있는 위치의 별도 배열의 요소 값을 1만큼 증가시킵니다.

23	41	25	87	33	96	26	74	51	84
8	4	7	2	5	1	6	3	3	2

9번째 반복 : 9번째 숫자인 51보다 작거나 같은 숫자가 있는 위치의 별도 배열의 요소 값을 1만큼 증가시킵니다.

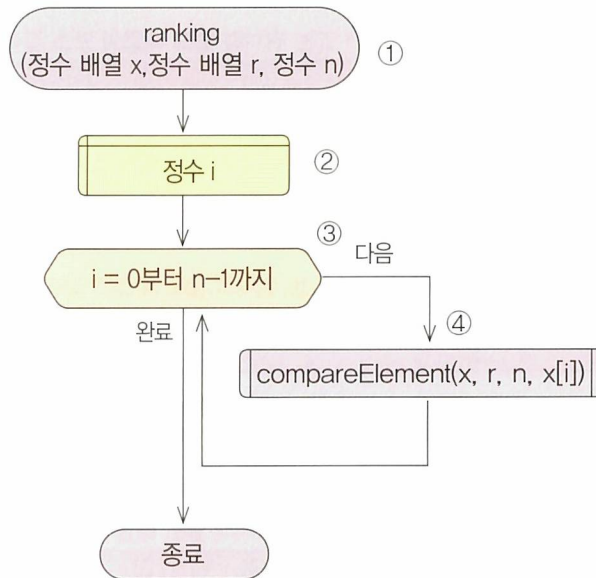
23	41	25	87	33	96	26	74	51	84
9	5	8	2	6	1	7	3	4	2

10번째 반복 : 10번째 숫자인 84보다 작거나 같은 숫자가 있는 위치의 별도 배열의 요소 값을 1만큼 증가시킵니다.

23	41	25	87	33	96	26	74	51	84
10	6	9	2	7	1	8	4	5	3



순서도와 알고리즘 해설(※ 145쪽 주프로그램 참고)



① 함수를 정의합니다.

- 함수명은 ranking입니다.
- 배열 x는 정수들이 0이 저장되어 있는 배열입니다.
- 배열 r은 주어진 숫자보다 작거나 같은가를 표시하기 위한 배열입니다.
- 변수 n은 배열의 크기를 지정합니다.

② 이 알고리즘에서 필요한 변수는 다음과 같습니다.

- 변수 i는 반복을 제어하기 위한 변수입니다.

③ 배열의 처음부터 끝까지 반복하도록 반복 횟수를 설정합니다.

- 변수 i의 값을 0부터 n-1까지 1씩 증가시키면서 반복합니다.

④ 배열의 모든 요소에 대하여 i번째 요소와 크기를 비교합니다.

- 크기를 비교한 결과를 순위를 지정하는 배열의 요소에 더합니다.(※76쪽 참고)

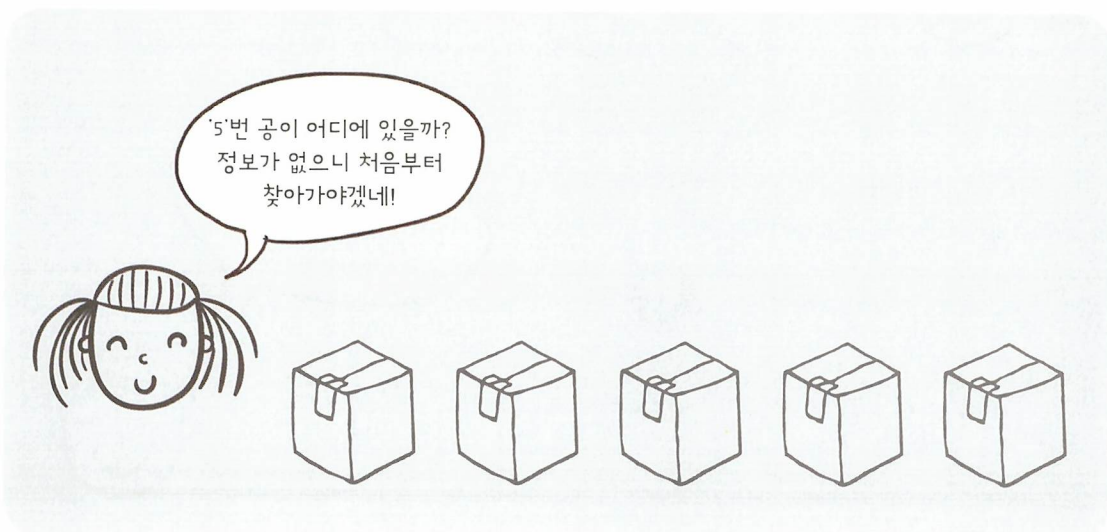


탐색하기-탐색의 개요 및 알고리즘 구조

탐색이란 컴퓨터에 저장되어 있는 많은 데이터들 중에서 주어진 데이터가 있는 위치를 찾아내는 것을 말합니다. 데이터가 저장되어 있는 상태, 즉 정렬되어 있는가 혹은 어떤 분포를 가지는가에 따라 효율적으로 탐색할 수 있는 알고리즘이 다릅니다. 기본적인 탐색 알고리즘으로는 순차 탐색과 이진 탐색이 있는데 탐색 알고리즘의 전형적인 형태는 [그림 5-3]과 같으며 “Search” 라고 하는 함수 대신에 다양한 탐색 함수로 대체하면 됩니다.

순차 탐색은 요소들이 나열되어 있을 때 처음 요소부터 시작하여 그 요소가 찾는 데이터인지 비교해 가는 방법입니다. 만약 찾는 데이터가 있으면 탐색이 성공한 것이고 마지막 요소까지 비교하였는데도 없으면 찾는 데이터가 없는 것입니다.

순차 탐색으로 데이터를 찾는 경우에 찾는 데이터가 처음에 위치하면 한 번만 비교해서 찾을 수도 있지만 찾는 데이터가 가장 끝에 위치하거나 없으면 모든 데이터를 비교해야만 합니다. 따라서, 탐색 시간의 관점에서 본다면 최악의 경우가 발생할 수도 있습니다. 그러나, 데이터가 정렬되어 있지 않은 경우와 같이 데이터 분포에 대한 정보가 전혀 없는 경우에는 이 방법만 사용할 수 있습니다. 순차 탐색의 개념을 그림으로 표현하면 다음과 같습니다.





2



'5'번 공이 아니네요.
다음 상자를
열어봅시다.



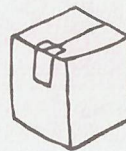
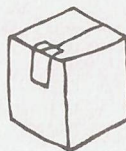
'5'번 공이 아니네요.
다음 상자를 열어봅시다.

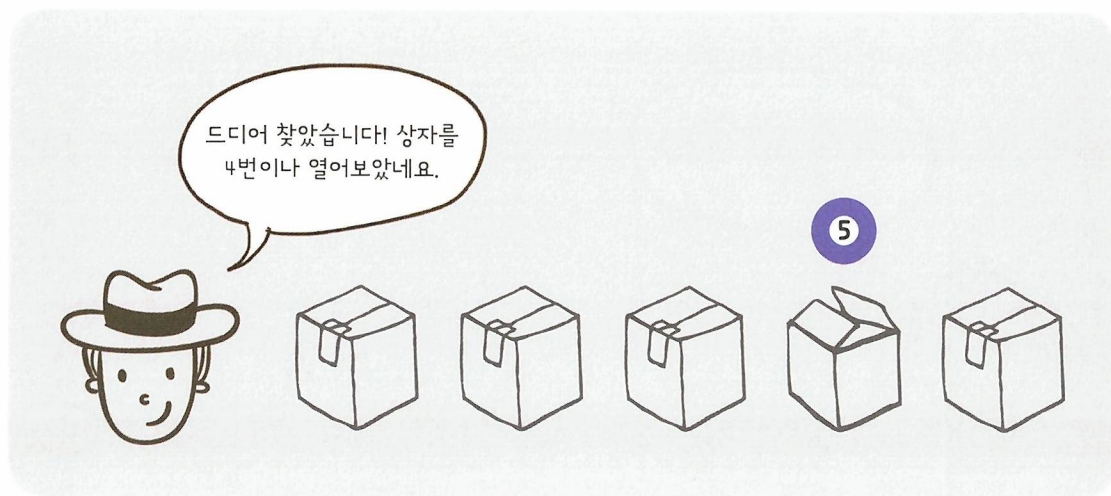
7



'5'번 공이 아니네요.
다음 상자를
열어봅시다.

4



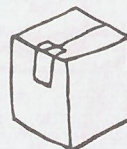
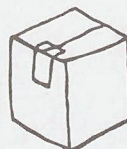
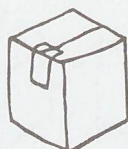


이진 탐색은 데이터들이 순서대로 나열되어 있을 때 사용할 수 있는 탐색 방법입니다. 즉, 나열되어 있는 데이터들 중에서 가운데에 있는 데이터가 찾는 데이터인지 제일 먼저 검사합니다. 만약 찾는 데이터라면 찾은 것이지만 다르다면 다음 동작으로 진행해야 합니다. 찾는 데이터가 가운데의 데이터보다 작다면 찾는 데이터는 가운데보다 앞쪽에 있다고 볼 수 있으므로 가운데보다 앞쪽에서 찾고, 찾는 데이터가 가운데의 데이터보다 크다면 가운데보다 뒤쪽에 있다고 볼 수 있으므로 가운데보다 뒤쪽에서 찾으면 될 것입니다. 이때도 각 부분의 가운데에 있는 데이터와 비교합니다. 이렇게 하면 순차 탐색보다 훨씬 효율적으로 탐색할 수 있게 됩니다. 단, 이진 탐색 방법을 사용하기 위해서는 반드시 데이터가 정렬되어 있어야 합니다.

이진 탐색의 개념을 그림으로 표현하면 다음과 같습니다.



'5'번 공이 어디에 있을까? 순서대로 되어있으니 가운데부터 찾아볼까요?



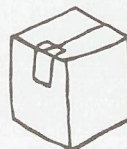
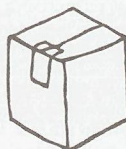
'5'보다 큰 공이네요.
그러면 앞쪽에서 찾아야 할 것 같습니다.

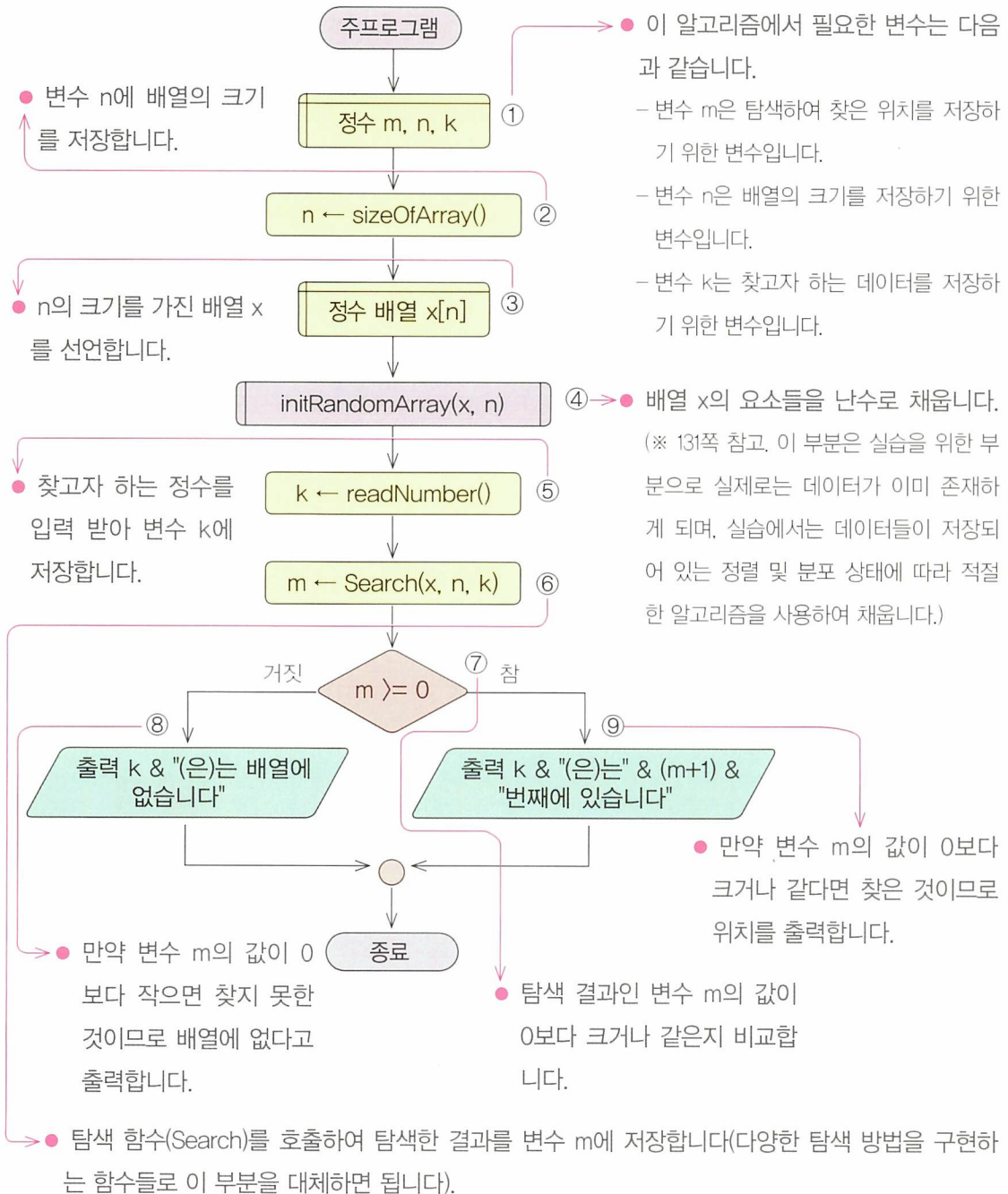
7



드디어 찾았습니다.
상자를 2번만 열어보고
찾았네요.

5





[그림 5-3] 탐색 알고리즘의 전형



문제 숫자들이 저장되어 있는 배열에서 주어진 숫자가 저장된 위치를 출력해 봅시다.
(순차 탐색)

문제 해결 방법

배열의 처음부터 끝까지 차례로 주어진 숫자와 비교하여 찾아가면 됩니다.

※ 관련 알고리즘 : n 개의 데이터 합 구하기 (62쪽 참고)

[입력] 숫자들이 저장되어 있는 배열, 찾고자 하는 숫자

[출력] 주어진 숫자가 있는 배열 요소의 위치

다음의 예를 통해 내용을 이해해 봅시다. 이때, 정수 51이 주어집니다. 이 배열에서 51은 9번째에 위치해 있습니다(※ 위치는 첨자에 1을 더합니다.).

배열									
23	41	25	87	33	96	26	74	51	84
↓ 주어진 숫자 : 51									
23	41	25	87	33	96	26	74	51	84
0	1	2	3	4	5	6	7	8	9

다음과 같은 순서로 문제를 해결할 수 있습니다.

- 주어진 숫자를 찾거나 더 이상 탐색할 요소가 없을 때까지 다음을 반복합니다.
 - 배열 요소의 위치를 다음으로 이동합니다.
- 탐색 결과를 반환합니다.



🔒 문제 해결의 예

|예| 10개의 요소를 가진 배열에서 51의 위치 찾기

1번째 반복 : $x[0]$ 은 23으로 51과 다르므로 다음으로 이동합니다.

23	41	25	87	33	96	26	74	51	84
0	1	2	3	4	5	6	7	8	9

2번째 반복 : $x[1]$ 은 41로 51과 다르므로 다음으로 이동합니다.

23	41	25	87	33	96	26	74	51	84
0	1	2	3	4	5	6	7	8	9

3번째 반복 : $x[2]$ 은 25로 51과 다르므로 다음으로 이동합니다.

23	41	25	87	33	96	26	74	51	84
0	1	2	3	4	5	6	7	8	9

4번째 반복 : $x[3]$ 은 87로 51과 다르므로 다음으로 이동합니다.

23	41	25	87	33	96	26	74	51	84
0	1	2	3	4	5	6	7	8	9

5번째 반복 : $x[4]$ 은 33으로 51과 다르므로 다음으로 이동합니다.

23	41	25	87	33	96	26	74	51	84
0	1	2	3	4	5	6	7	8	9

6번째 반복 : $x[5]$ 은 96으로 51과 다르므로 다음으로 이동합니다.

23	41	25	87	33	96	26	74	51	84
0	1	2	3	4	5	6	7	8	9

7번째 반복 : $x[6]$ 은 26으로 51과 다르므로 다음으로 이동합니다.

23	41	25	87	33	96	26	74	51	84
0	1	2	3	4	5	6	7	8	9

8번째 반복 : $x[7]$ 은 74로 51과 다르므로 다음으로 이동합니다.

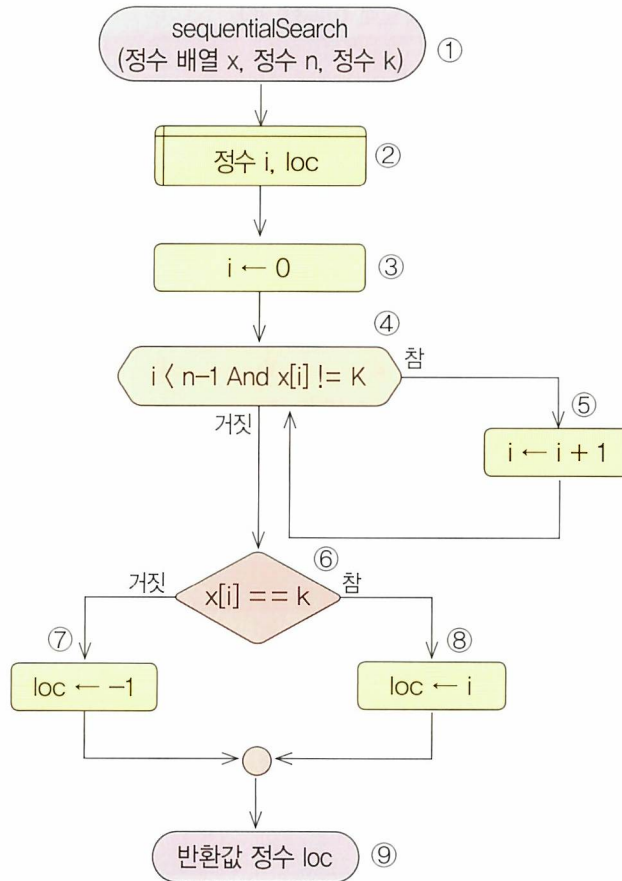
23	41	25	87	33	96	26	74	51	84
0	1	2	3	4	5	6	7	8	9

9번째 반복 : $x[8]$ 의 51이 51과 같으므로 찾고자 하는 요소가 있는 첨자는 8, 위치는 9번째입니다.

23	41	25	87	33	96	26	74	51	84
0	1	2	3	4	5	6	7	8	9



순서도와 알고리즘 해설(※ 146쪽 주프로그램 참고)





① 함수를 정의합니다.

- 함수명은 sequentialSearch입니다.
- 배열 x는 정수들이 저장되어 있는 배열입니다.
- 변수 n은 배열의 크기를 지정합니다.
- 변수 k에는 찾고자 하는 숫자가 저장되어 있습니다.

② 이 알고리즘에서 필요한 변수는 다음과 같습니다.

- 변수 i는 반복을 제어하기 위한 변수입니다.
- 변수 loc는 찾고자 하는 숫자가 있는 배열의 첨자를 저장합니다.

③ 배열의 위치를 지정하고 반복 횟수를 제어하는 변수를 초기화합니다.

- 배열의 첫 번째 요소를 가리키고 반복을 시작하기 위하여 0으로 초기화합니다.

④ 반복 조건을 설정합니다.

- 배열의 끝에 도달하지 못했거나 찾고자 하는 숫자를 찾지 못했다면 계속 반복을 수행합니다.

⑤ 배열의 다음 요소를 가리키고 반복을 계속하기 위하여 변수의 값을 변화시킵니다.

- 변수 i는 배열의 첨자와 반복 제어의 역할을 동시에 합니다. 따라서 다음 요소를 가리키고 반복을 계속하기 위하여 1만큼 증가시킵니다.

⑥ 반복이 끝난 다음에 찾고자 하는 숫자를 찾은 것인지 아닌지 판단합니다.

- 찾고자 하는 숫자를 찾아서 반복을 끝낸 것인지 아니면 배열에 찾고자 하는 숫자가 없어서 끝에 도달한 것인지 판단하기 위하여 배열의 현재 요소와 찾고자 하는 숫자를 비교합니다.

⑦ 찾고자 하는 숫자를 찾지 못한 경우로 표시합니다.

- 배열의 현재 요소와 찾고자 하는 숫자가 같지 않다면 찾지 못한 것이므로 변수 loc에 -1을 저장합니다.

⑧ 찾고자 하는 숫자가 있는 위치를 저장합니다.

- 배열의 현재 요소와 찾고자 하는 숫자가 같다면 찾은 것이므로 변수 loc에 현재 위치인 변수 i의 값을 저장합니다.

⑨ 탐색 결과를 반환합니다.

- 변수 loc의 값을 반환합니다.



문제 숫자가 순서대로 저장되어 있는 배열에서 주어진 숫자가 저장된 위치를 출력해 봅시다. (이진 탐색)

문제 해결 방법

배열의 가운데에 있는 숫자와 먼저 비교합니다. 만약 찾고자 하는 숫자가 가운데에 있는 숫자보다 작다면 가운데보다 앞쪽에서 찾고, 가운데에 있는 숫자보다 크다면 가운데보다 뒤쪽에서 찾습니다. 같다면 저장된 위치를 찾은 것이 됩니다. 이와 같은 동작을 반복하다 보면 앞에서부터 차례로 비교하는 것보다 주어진 숫자를 빠르게 찾을 수 있습니다.

※ 관련 알고리즘 : 2개의 숫자를 비교하여 작은 숫자 찾기 (45쪽 참고)

[입력] 숫자들이 정렬되어 저장되어 있는 배열, 찾고자 하는 숫자

[출력] 찾고자 하는 숫자가 있는 배열 요소의 위치

다음의 예를 통해 내용을 이해해 봅시다. 이때, 숫자 23이 주어집니다. 이 배열에서 23은 2번째에 있습니다. (위치는 첨자에 1을 더합니다.)

배열									
17	23	26	33	41	51	74	84	87	96

↓ 주어진 숫자 : 23

17	23	26	33	41	51	74	84	87	96
0	1	2	3	4	5	6	7	8	9

다음과 같은 순서로 문제를 해결할 수 있습니다.

1. 찾고자 하는 숫자를 찾거나 더 이상 탐색할 숫자가 없을 때까지 다음을 반복합니다.
 - 1.1 만약 찾고자 하는 숫자가 배열의 가운데 요소보다 크다면
 - 1.1.1 배열에서 가운데보다 뒤쪽을 탐색합니다.
 - 1.2 그렇지 않다면
 - 1.2.1 배열에서 가운데보다 앞쪽을 탐색합니다.
2. 탐색 결과를 반환합니다.



🔒 문제 해결의 예

예 10개의 요소를 가진 배열에서 26의 위치 찾기

(※ 여기에서 변수 l은 탐색하고자 하는 구간의 시작 첨자, h는 끝 첨자, m은 가운데를 가리킵니다.)

1번째 반복 : 탐색하고자 하는 구간의 시작 첨자를 저장하고 있는 변수 l의 값이 0, 끝 첨자를 저장하고 있는 변수 h의 값이 9이기 때문에 중간값(m)이 4가 되므로 첨자가 4인 위치의 요소 값인 41을 찾고자 하는 숫자인 26과 비교하면 찾는 데이터는 첨자가 4인 위치보다 앞쪽에 있을 것으로 추정됩니다.

17	23	26	33	41	51	74	84	87	96
0	1	2	3	4	5	6	7	8	9

2번째 반복 : 찾고자 하는 데이터가 첨자가 4인 곳보다 앞에 있을 것으로 추정되므로 탐색 구간은 변수 l의 값은 그대로 두고, 변수 h의 값을 가운데인 4보다 1만큼 작은 3으로 정하여 설정할 수 있습니다. 이 구간의 중간값(m)은 1이 되므로 첨자가 1인 위치의 요소 값인 23을 26과 비교하면 되는데, 비교한 결과 26이 23보다 크므로 찾고자 하는 데이터는 첨자가 1인 위치보다 뒤쪽에 있을 것으로 추정됩니다.

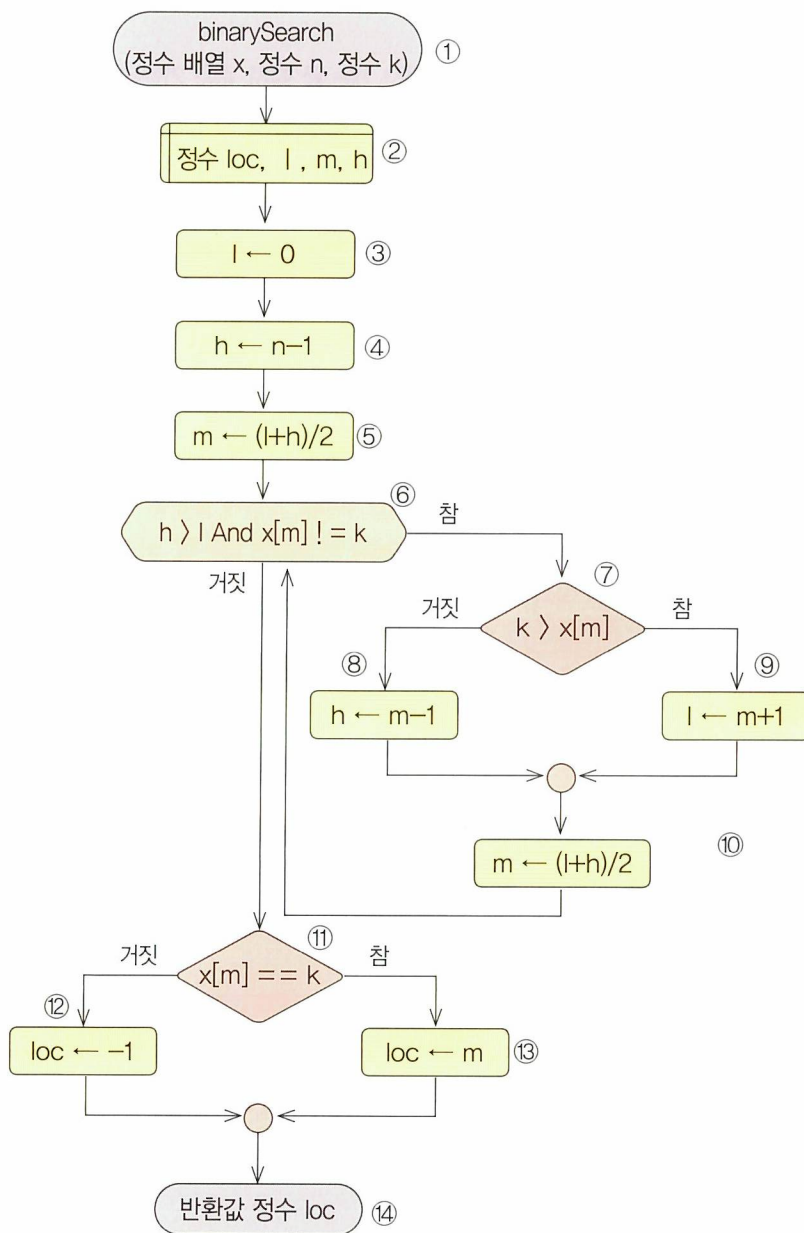
17	23	26	33	41	51	74	84	87	96
0	1	2	3	4	5	6	7	8	9

3번째 반복 : 찾고자 하는 데이터가 첨자가 1인 곳보다 뒤쪽에 있을 것으로 추정되므로 탐색 구간은 변수 h의 값은 그대로 두고, 변수 l의 값을 가운데인 1보다 1만큼 큰 2로 정하여 설정할 수 있습니다. 이 구간의 중간값(m)은 2가 되므로 첨자가 2인 위치의 요소 값인 26과 비교하면 되는데, 비교한 결과 같은 숫자이므로 데이터를 찾은 것이 됩니다.

17	23	26	33	41	51	74	84	87	96
0	1	2	3	4	5	6	7	8	9



순서도와 알고리즘 해설(※ 148쪽 주프로그램 참고)





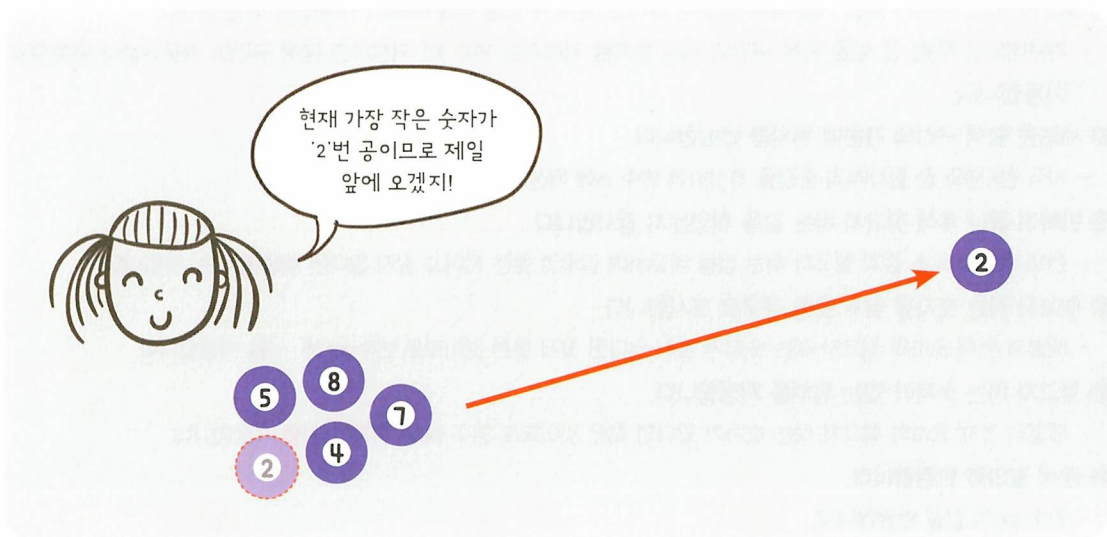
- ① 함수를 정의합니다.
 - 함수명은 `binarySearch`입니다. - 배열 `x`는 숫자들이 순서대로 저장되어 있는 배열입니다.
 - 변수 `n`은 배열의 크기를 지정합니다. - 변수 `k`는 찾고자 하는 숫자를 저장합니다.
- ② 이 알고리즘에서 필요한 변수는 다음과 같습니다.
 - 변수 `loc`는 찾는 숫자가 있는 배열의 첨자를 저장합니다.
 - 변수 `l`, `m`, `h`은 탐색 구간의 시작 첨자, 가운데, 끝 첨자를 저장하기 위한 변수들입니다.
- ③ 처음 탐색 구간의 시작 첨자를 초기화합니다.
 - 탐색 구간의 작은 쪽을 가리키는 변수 `l`에 0을 저장합니다.
- ④ 처음 탐색 구간의 끝 첨자를 초기화합니다.
 - 탐색 구간의 끝 첨자를 가리키는 변수 `h`에 `n-1`을 저장합니다.
- ⑤ 탐색 구간의 가운데를 계산합니다.
 - 가운데의 첨자를 저장하는 변수 `m`에 변수 `l`과 변수 `h` 값의 중간을 계산하여 저장합니다.
- ⑥ 반복 조건을 설정합니다.
 - 시작 첨자가 끝 첨자보다 작고, 아직 찾고자 하는 숫자를 찾지 못하였으면 계속 반복합니다.
- ⑦ 찾고자 하는 숫자와 현재 위치의 숫자를 비교합니다.
 - 찾고자 하는 숫자를 저장하고 있는 변수 `k`와 탐색 구간의 가운데 요소인 `x[m]`의 값을 비교합니다.
- ⑧ 찾고자 하는 숫자가 배열 가운데의 값보다 크지 않다면 탐색 구간을 가운데보다 앞쪽으로 이동합니다.
 - 가운데보다 1만큼 작은 값을 탐색 구간의 끝 첨자를 가리키는 변수 `h`에 저장하면 탐색 구간이 가운데보다 앞쪽으로 이동합니다.
- ⑨ 찾고자 하는 숫자가 배열 가운데의 값보다 크다면 탐색 구간을 가운데보다 뒤쪽으로 이동합니다.
 - 가운데보다 1만큼 큰 값을 탐색 구간의 시작 첨자를 가리키는 변수 `l`에 저장하면 탐색 구간이 가운데보다 뒤쪽으로 이동합니다.
- ⑩ 새로운 탐색 구간의 가운데 위치를 연산합니다.
 - 시작 첨자(`l`)와 끝 첨자(`h`)의 중간을 연산하여 변수 `m`에 저장합니다.
- ⑪ 반복이 끝난 후에 찾고자 하는 값을 찾았는지 검사합니다.
 - 현재 위치의 요소 값과 찾고자 하는 값을 비교하여 같으면 찾은 것이고 같지 않다면 배열에 없는 것입니다.
- ⑫ 찾고자 하는 숫자를 찾지 못한 경우로 표시합니다.
 - 배열의 현재 요소와 찾고자 하는 숫자가 같지 않다면 찾지 못한 것이므로 변수 `loc`에 `-1`을 저장합니다.
- ⑬ 찾고자 하는 숫자가 있는 위치를 저장합니다.
 - 배열의 현재 요소와 찾고자 하는 숫자가 같다면 찾은 것이므로 변수 `loc`에 현재 위치를 저장합니다.
- ⑭ 탐색 결과를 반환합니다.
 - 변수 `loc`의 값을 반환합니다.



정렬하기-정렬의 개요 및 알고리즘 구조

정렬이란 데이터들을 순서대로 나열하는 것을 말합니다. 순서대로 나열하는 종류에는 오름차순 및 내림차순 정렬이 있습니다. 오름차순 정렬은 작은 것부터 큰 것 순으로 정렬하는 것이고, 내림차순 정렬은 큰 것부터 작은 것 순으로 정렬하는 것을 말합니다. 정렬 방법에는 다양한 방법이 있는데 이 책에서는 기본적인 방법인 선택, 삽입, 버블 정렬 알고리즘에 대하여 소개하겠습니다. 정렬 알고리즘의 전형적인 형태는 [그림 5-4]와 같으며 “Sort” 라고 하는 함수 대신에 다양한 정렬 알고리즘 함수로 대체하면 됩니다.

선택 정렬은 현재 남아있는 데이터들 중에서 가장 작은 데이터를 찾아 차례대로 나열하는 방법입니다. 즉, 현재의 데이터들 중에서 가장 작은 데이터를 찾아 정렬되어 있는 데이터들의 가장 끝에 추가하는 것을 반복하면 모든 데이터들이 순서대로 나열되는 원리입니다. 데이터들 중에서 가장 작은 데이터를 찾는 알고리즘은 ‘가장 작은 숫자 찾기(※ 79쪽 참고)’에서 설명했으므로 이를 모든 데이터들에 대해서 반복하면 됩니다. 선택 정렬의 개념을 그림으로 표현하면 다음과 같습니다.

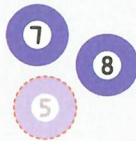




현재 가장 작은 숫자가
'4'번 공이므로 다음에
추가하면 되겠지!



현재 가장 작은 숫자가
'5'번 공이므로 다음에
추가하면 되겠지!



현재 가장 작은 숫자가
'7'번 공이므로 다음에
추가하면 되겠지!



현재 가장 작은 숫자가
'8'번 공이므로 다음에
추가하면 되겠지!

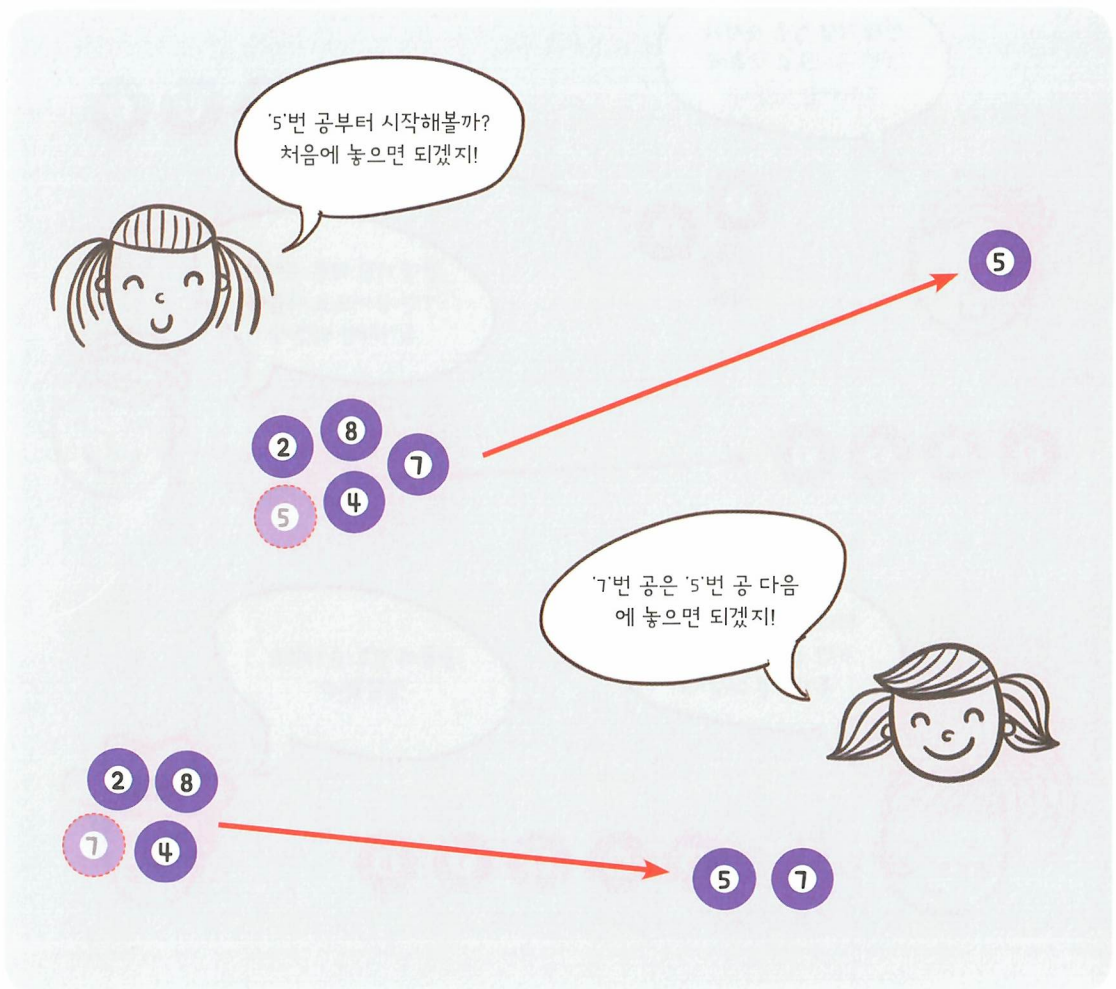


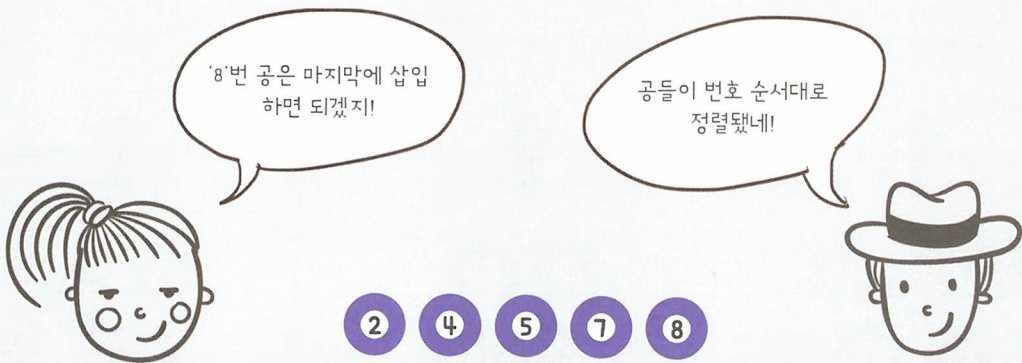
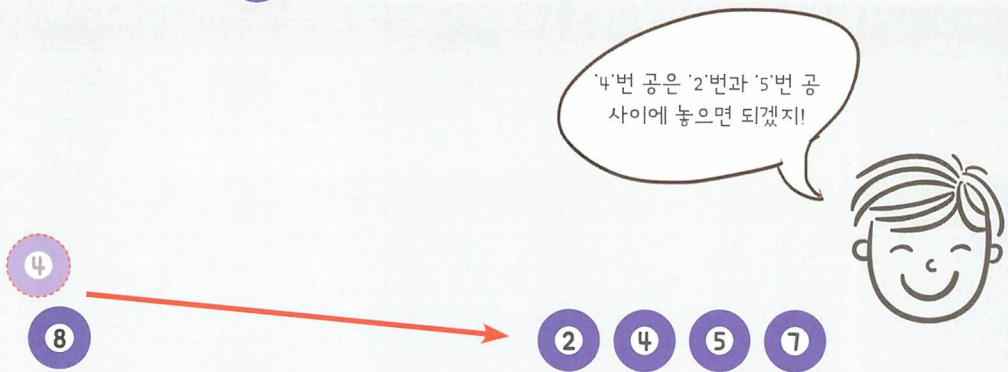
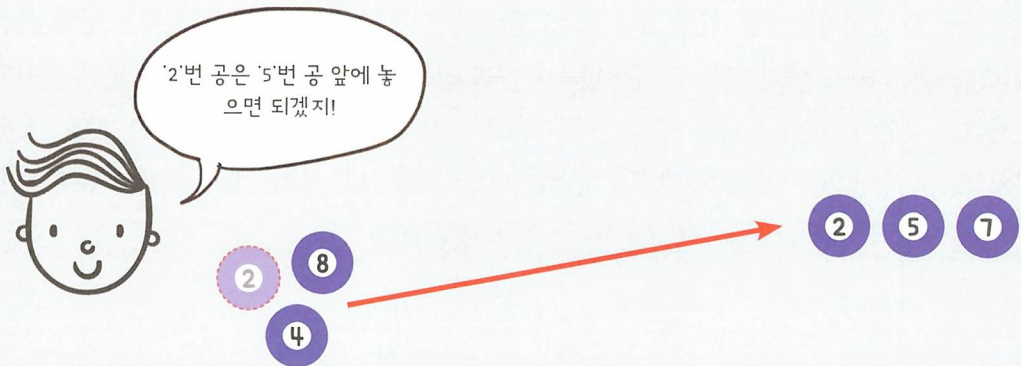
공들이 번호 순서대로
정렬됐네!





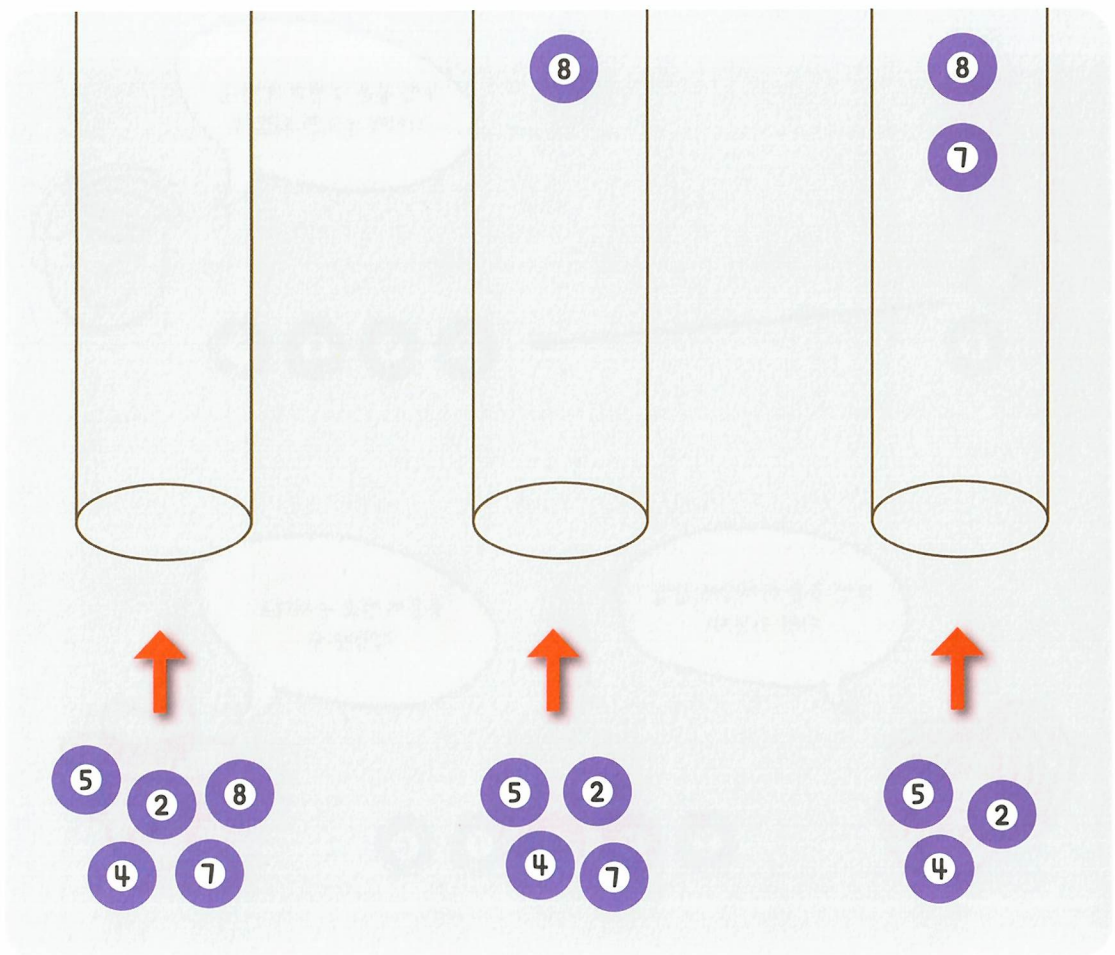
삽입 정렬은 현재 남아있는 데이터들 중에서 하나를 선택하여 정렬되어 있는 데이터들 속에 순서에 맞게 삽입하여 정렬하는 방법입니다. 즉, 현재의 데이터들 중에서 데이터 하나를 선택하여 정렬되어 있는 데이터들 속에 순서에 맞게 삽입하는 것을 반복하면 모든 데이터들이 순서대로 나열되는 원리입니다. 주어진 데이터를 정렬되어 있는 데이터들 속에 삽입하는 알고리즘은 ‘순서에 맞게 삽입하기(※ 65쪽 참고)’에서 설명했으므로 이를 모든 데이터들에 대해서 반복하면 됩니다. 삽입 정렬의 개념을 그림으로 표현하면 다음과 같습니다.

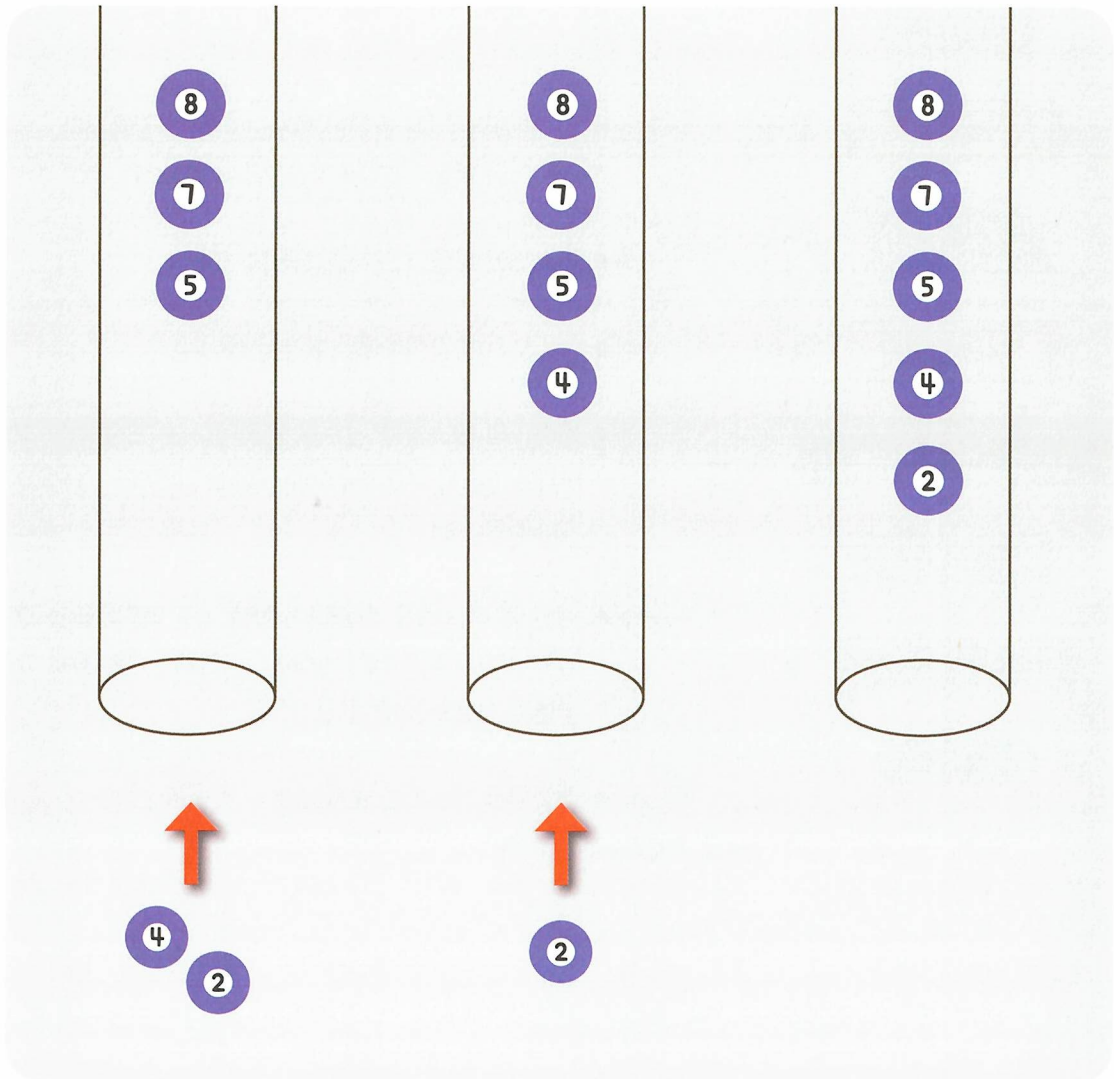


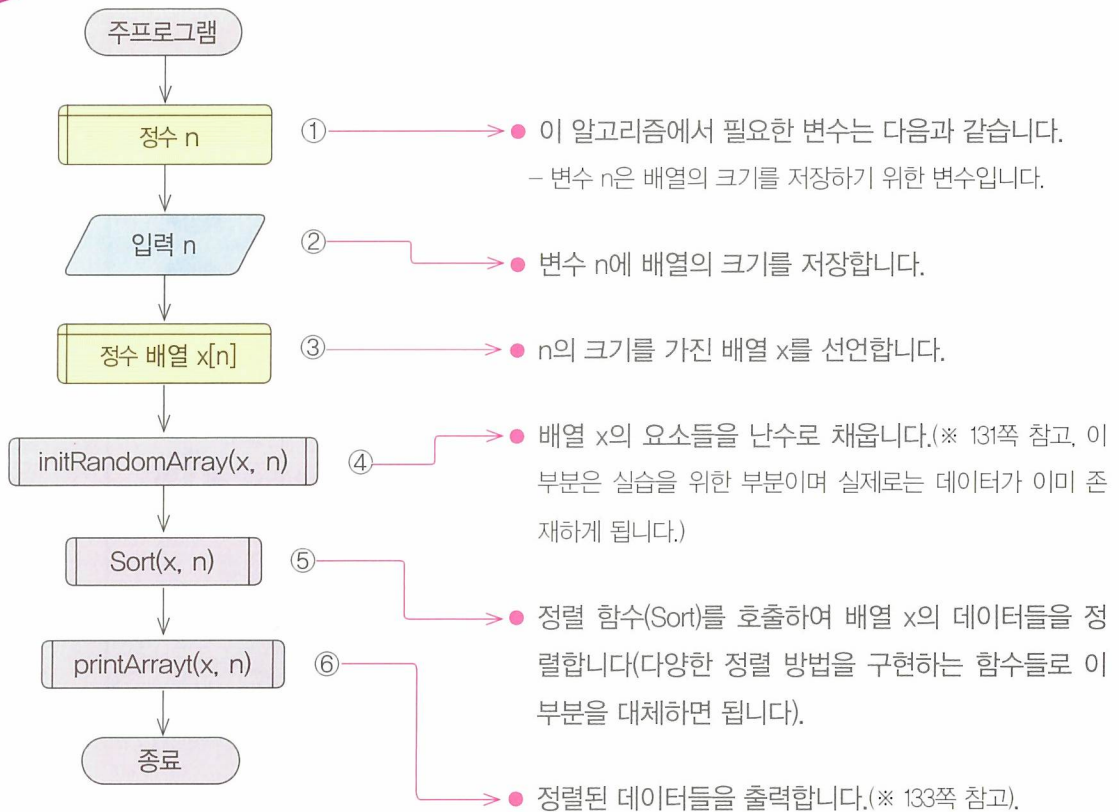




버블 정렬은 물에 뜨는 힘이 강한 방울일수록 가장 먼저 위로 떠오르게 되는 것처럼 현재 데이터들 중에서 가장 큰 데이터를 위로 띄워(실제 프로그램에서는 마지막으로 보냄) 정렬하는 방법입니다. 즉, 현재의 데이터들 중에서 가장 큰 데이터를 마지막으로 보내는 동작을 반복하면 모든 데이터들이 순서대로 나열되는 원리입니다. 가장 큰 데이터를 마지막으로 보내는 알고리즘은 ‘가장 큰 숫자를 마지막으로 보내기(※ 82쪽 참고)’에서 설명했으므로 이를 모든 데이터들에 대해서 반복하면 됩니다. 버블 정렬의 개념을 그림으로 표현하면 다음과 같습니다.







[그림 5-4] 정렬 알고리즘의 전형



문제 배열에 저장되어 있는 숫자들을 선택 정렬 알고리즘을 사용하여 정렬하여 봅시다.
(선택 정렬)

🕒 문제 해결 방법

남아 있는 숫자들 중에서 가장 작은 값을 찾아 정렬되어 있는 배열의 끝에 위치시키는 것을 반복하면 전체를 정렬할 수 있습니다.

※ 관련 알고리즘 : 가장 작은 숫자 찾기 (79쪽 참고) / 2개의 숫자를 서로 바꾸기 (26쪽 참고)

[입력] 숫자들이 저장되어 있는 배열

[출력] 숫자들이 정렬되어 저장된 배열

다음의 예를 통해 내용을 이해해 봅시다.

배열									
23	41	17	87	33	96	26	74	51	84
↓									
결과									
17	23	26	33	41	51	74	84	87	96

다음과 같은 순서로 문제를 해결할 수 있습니다.

- 남아있는 배열의 크기를 하나씩 줄여 가면서 다음을 반복합니다.
 - 가장 작은 숫자를 찾아 정렬되어 있는 배열의 마지막에 위치시킵니다.

 문제 해결의 예

|예| 10개의 요소를 가진 배열-선택 정렬

처음 배열 : 정렬하고자 하는 배열 전체의 범위는 첨자가 0부터 9까지입니다.

23	41	17	87	33	96	26	74	51	84
----	----	----	----	----	----	----	----	----	----

1번째 반복 : 첨자가 0부터 9까지인 범위에서 가장 작은 값인 17을 찾아 첨자가 0인 곳의 숫자인 23과 바꿉니다.

17	41	23	87	33	96	26	74	51	84
----	----	----	----	----	----	----	----	----	----

2번째 반복 : 첨자가 1부터 9까지인 범위에서 가장 작은 값인 23을 찾아 첨자가 1인 곳의 숫자인 41과 바꿉니다.

17	23	41	87	33	96	26	74	51	84
----	----	----	----	----	----	----	----	----	----

3번째 반복 : 첨자가 2부터 9까지인 범위에서 가장 작은 값인 26을 찾아 첨자가 2인 곳의 숫자인 41과 바꿉니다.

17	23	26	87	33	96	41	74	51	84
----	----	----	----	----	----	----	----	----	----

4번째 반복 : 첨자가 3부터 9까지인 범위에서 가장 작은 숫자인 33을 찾아 첨자가 3인 곳의 숫자인 87과 바꿉니다.

17	23	26	33	87	96	41	74	51	84
----	----	----	----	----	----	----	----	----	----

5번째 반복 : 첨자가 4부터 9까지인 범위에서 가장 작은 숫자인 41을 찾아 첨자가 4인 곳의 숫자인 87과 바꿉니다.

17	23	26	33	41	96	87	74	51	84
----	----	----	----	----	----	----	----	----	----

6번째 반복 : 첨자가 5부터 9까지인 범위에서 가장 작은 숫자인 51을 찾아 첨자가 5인 곳의 숫자인 96과 바꿉니다.

17	23	26	33	41	51	87	74	96	84
----	----	----	----	----	----	----	----	----	----

7번째 반복 : 첨자가 6부터 9까지인 범위에서 가장 작은 숫자인 74를 찾아 첨자가 6인 곳의 숫자인 87과 바꿉니다.

17	23	26	33	41	51	74	87	96	84
----	----	----	----	----	----	----	----	----	----

8번째 반복 : 첨자가 7부터 9까지인 범위에서 가장 작은 숫자인 84를 찾아 첨자가 7인 곳의 숫자인 87과 바꿉니다.

17	23	26	33	41	51	74	84	96	87
----	----	----	----	----	----	----	----	----	----

9번째 반복 : 첨자가 8부터 9까지인 범위에서 가장 작은 숫자인 87을 찾아 첨자가 8인 곳의 숫자인 96과 바꿉니다.

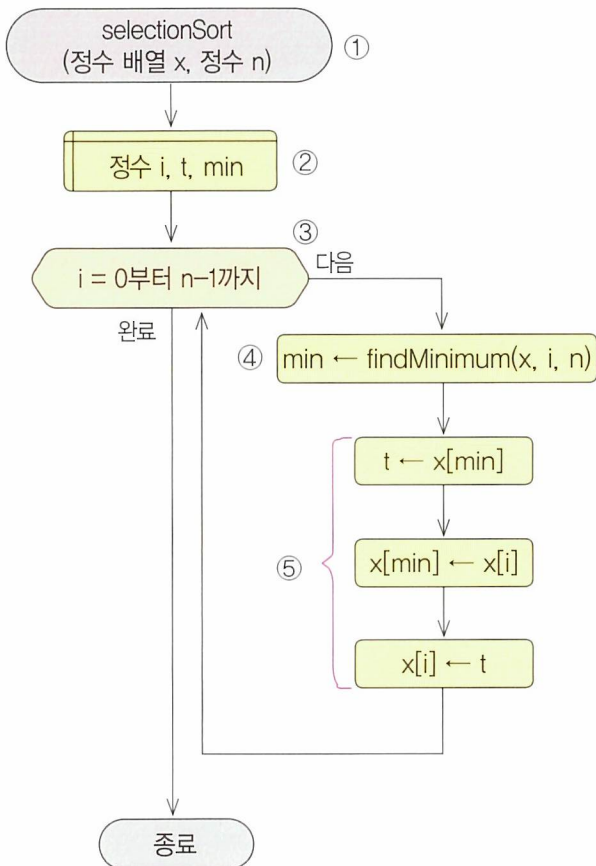
17	23	26	33	41	51	74	84	87	96
----	----	----	----	----	----	----	----	----	----

결과

17	23	26	33	41	51	74	84	87	96
----	----	----	----	----	----	----	----	----	----



순서도와 알고리즘 해설(※ 150쪽 주프로그램 참고)



① 함수를 정의합니다.

- 함수명은 selectionSort입니다.
- 배열 x에는 정수들이 저장되어 있습니다.
- 변수 n은 배열의 크기를 지정합니다.

② 이 알고리즘에서 필요한 변수는 다음과 같습니다.

- 변수 i는 반복을 제어하는데 사용됩니다.
- 변수 i는 위치를 가리키는 두 숫자를 바꾸는데 필요한 임시 변수입니다.
- 변수 min은 현재까지 가장 작은 요소의 위치를 저장합니다.

③ 배열의 처음부터 끝까지 반복할 수 있도록 반복 횟수를 설정합니다.

- 변수 i의 값을 0부터 n-1까지 1씩 증가시키면서 반복합니다. 예를 들어, 변수 n의 값이 10이면 0부터 9까지 10번 반복하게 됩니다.

④ 주어진 배열에서 가장 작은 숫자가 있는 위치를 가져옵니다.

- i부터 시작하는 배열에서 가장 작은 요소가 저장되어 있는 위치를 가져옵니다(※ 79쪽 참고).

⑤ 현재 위치의 값과 가장 작은 값을 서로 교환합니다.

- (※ 26쪽 참고).



문제 배열에 저장되어 있는 숫자들을 삽입 정렬 알고리즘을 사용하여 정렬하여 봅시다.
(삽입 정렬)

🕒 문제 해결 방법

배열의 요소를 앞에서부터 하나씩 선택하여 현재까지 정렬되어 있는 부분에 순서에 맞게 삽입하는 동작을 반복하면 전체를 정렬할 수 있습니다.

※ 관련 알고리즘 : 순서에 맞게 삽입하기 (65쪽 참고)

[입력] 숫자들이 저장되어 있는 배열

[출력] 숫자들이 정렬되어 저장된 배열

다음의 예를 통해 내용을 이해해 봅시다.

배열									
23	41	17	87	33	96	26	74	51	84
↓									
결과									
17	23	26	33	41	51	74	84	87	96

다음과 같은 순서로 문제를 해결할 수 있습니다.

- 2번째부터 끝까지의 요소에 대하여 차례로 다음을 반복합니다.
 - 현재까지 정렬된 배열에 순서에 맞게 요소를 삽입합니다.



🔒 문제 해결의 예

|예| 10개의 요소를 가진 배열-삽입 정렬

처음 배열

23	41	17	87	33	96	26	74	51	84
----	----	----	----	----	----	----	----	----	----

1번째 반복 : 2번째 요소를 첨자가 0~1인 구간에 순서에 맞게 삽입합니다.

23	41	17	87	33	96	26	74	51	84
----	----	----	----	----	----	----	----	----	----

2번째 반복 : 3번째 요소를 첨자가 0~2인 구간에 순서에 맞게 삽입합니다.

17	23	41	87	33	96	26	74	51	84
----	----	----	----	----	----	----	----	----	----

3번째 반복 : 4번째 요소를 첨자가 0~3인 구간에 순서에 맞게 삽입합니다.

17	23	41	87	33	96	26	74	51	84
----	----	----	----	----	----	----	----	----	----

4번째 반복 : 5번째 요소를 첨자가 0~4인 구간에 순서에 맞게 삽입합니다.

17	23	33	41	87	96	26	74	51	84
----	----	----	----	----	----	----	----	----	----

5번째 반복 : 6번째 요소를 첨자가 0~5인 구간에 순서에 맞게 삽입합니다.

17	23	33	41	87	96	26	74	51	84
----	----	----	----	----	----	----	----	----	----

6번째 반복 : 7번째 요소를 첨자가 0~6인 구간에 순서에 맞게 삽입합니다.

17	23	26	33	41	87	96	74	51	84
----	----	----	----	----	----	----	----	----	----

7번째 반복 : 8번째 요소를 첨자가 0~7인 구간에 순서에 맞게 삽입합니다.

17	23	26	33	41	74	87	96	51	84
----	----	----	----	----	----	----	----	----	----

8번째 반복 : 9번째 요소를 첨자가 0~8인 구간에 순서에 맞게 삽입합니다.

17	23	26	33	41	51	74	87	96	84
----	----	----	----	----	----	----	----	----	----

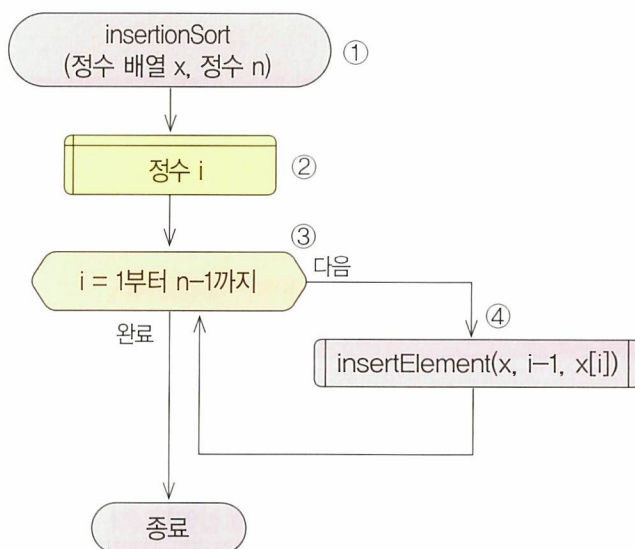
9번째 반복 : 10번째 요소를 첨자가 0~9인 구간에 순서에 맞게 삽입합니다.

17	23	26	33	41	51	74	84	87	96
----	----	----	----	----	----	----	----	----	----

결과

17	23	26	33	41	51	74	84	87	96
----	----	----	----	----	----	----	----	----	----

순서도와 알고리즘 해설(※ 151쪽 주프로그램 참고)



① 함수를 정의합니다.

- 함수명은 insertionSort입니다.
- 배열 x는 정수들이 저장되어 있는 배열입니다.
- 변수 n은 배열의 크기를 지정합니다.

② 알고리즘에서 필요한 변수는 다음과 같습니다.

- 변수 i는 반복을 제어하는데 사용합니다.

③ 배열의 2번째부터 끝까지 반복할 수 있도록 반복 횟수를 설정합니다.

- 변수 i의 값을 1부터 끝(n-1)까지 1씩 증가시키면서 반복합니다.
- 변수 i의 값을 2번째에 해당하는 1부터 시작하는 이유는 첫 번째인 경우에는 요소가 하나이므로 순서대로 할 필요가 없기 때문입니다.

④ i+1번째 숫자를 순서에 맞게 삽입합니다.

- 첨자가 i인 배열 요소의 값을 지금까지 정렬된 숫자 배열(첨자가 0부터 i-1까지인 배열)에 순서에 맞게 삽입합니다.



문제 배열에 저장되어 있는 숫자들을 버블 정렬 알고리즘을 사용하여 정렬하여 봅시다.
(버블 정렬)

🕒 문제 해결 방법

가장 큰 숫자를 배열의 마지막으로 보내는 동작을 남아있는 배열의 크기를 하나씩 줄여 가면서 수행하면 전체를 정렬할 수 있습니다.

※ 관련 알고리즘 : 가장 큰 숫자를 마지막으로 보내기 (82쪽 참고)

[입력] 숫자들이 저장되어 있는 배열

[출력] 숫자들이 정렬되어 저장된 배열

다음의 예를 통해 내용을 이해해 봅시다.

배열									
23	41	25	87	33	96	26	74	51	84
↓									
결과									
23	25	26	33	41	51	74	84	87	96

다음과 같은 순서로 문제를 해결할 수 있습니다.

- 남아있는 배열의 크기를 하나씩 줄여 가면서 다음을 반복합니다.
 - 가장 큰 숫자를 배열의 마지막으로 보냅니다.



문제 해결의 예

[예] 10개의 요소를 가진 배열-버블 정렬**1번째 반복 :** 첨자가 0~9인 범위에서 가장 큰 값인 96을 마지막(첨자 : 9)에 저장합니다.

23	25	41	33	87	26	74	51	84	96
----	----	----	----	----	----	----	----	----	----

2번째 반복 : 첨자가 0~8인 범위에서 가장 큰 값인 87을 마지막(첨자 : 8)에 저장합니다.

23	25	33	41	26	74	51	84	87	96
----	----	----	----	----	----	----	----	----	----

3번째 반복 : 첨자가 0~7인 범위에서 가장 큰 값인 84를 마지막(첨자 : 7)에 저장합니다.

23	25	33	26	41	51	74	84	87	96
----	----	----	----	----	----	----	----	----	----

4번째 반복 : 첨자가 0~6인 범위에서 가장 큰 값인 74를 마지막(첨자 : 6)에 저장합니다.

23	25	26	33	41	51	74	84	87	96
----	----	----	----	----	----	----	----	----	----

5번째 반복 : 첨자가 0~5인 범위에서 가장 큰 값인 51을 마지막(첨자 : 5)에 저장합니다.

23	25	26	33	41	51	74	84	87	96
----	----	----	----	----	----	----	----	----	----

6번째 반복 : 첨자가 0~4인 범위에서 가장 큰 값인 41을 마지막(첨자 : 4)에 저장합니다.

23	25	26	33	41	51	74	84	87	96
----	----	----	----	----	----	----	----	----	----

7번째 반복 : 첨자가 0~3인 범위에서 가장 큰 값인 33을 마지막(첨자 : 3)에 저장합니다.

23	25	26	33	41	51	74	84	87	96
----	----	----	----	----	----	----	----	----	----

8번째 반복 : 첨자가 0~2인 범위에서 가장 큰 값인 26을 마지막(첨자 : 2)에 저장합니다.

23	25	26	33	41	51	74	84	87	96
----	----	----	----	----	----	----	----	----	----

9번째 반복 : 첨자가 0~1인 범위에서 가장 큰 값인 25를 마지막(첨자 : 1)에 저장합니다.

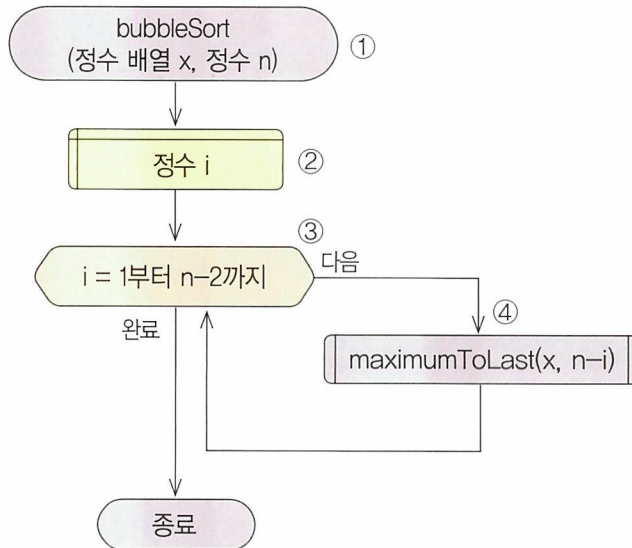
23	25	26	33	41	51	74	84	87	96
----	----	----	----	----	----	----	----	----	----

결과

23	25	26	33	41	51	74	84	87	96
----	----	----	----	----	----	----	----	----	----



순서도와 알고리즘 해설(※ 152쪽 주프로그램 참고)



① 함수를 정의합니다.

- 함수명은 bubbleSort입니다.
- 배열 x는 정수들이 저장되어 있는 배열입니다.
- 변수 n은 배열의 크기를 지정합니다.

② 이 알고리즘에서 필요한 변수는 다음과 같습니다.

- 변수 i는 반복을 제어하는데 사용합니다.

③ 배열의 처음부터 마지막의 하나 이전까지 반복할 수 있도록 반복 횟수를 설정합니다.

- 변수 i의 값을 0부터 n-2까지 1씩 증가시키면서 반복합니다. 예를 들어, 변수 n의 값이 10이면 0부터 8까지 9번 반복하게 됩니다.
- 반복을 마지막까지 하지 않는 이유는 다음 요소와 비교해야 하는데 마지막 요소의 다음 요소는 없기 때문입니다.

④ 주어진 배열에서 가장 큰 숫자를 배열의 마지막으로 보냅니다.(※ 82쪽 참고)

- 크기가 (n-i)인 배열에 대하여 가장 큰 값을 마지막으로 보냅니다.
- 변수 i의 값이 0인 경우에 n-i는 10(10-0)이 되고, 변수 i의 값이 1이 되면 9(10-1)가 되어 처리되는 배열의 크기가 1씩 줄어들게 됩니다.